

ArtsyPhartsy Bag Uploader

Technical Reference & Design Guide

Version 3 — June 2026

[github.com/\[your-username\]/artsy-phartsy-uploader](https://github.com/[your-username]/artsy-phartsy-uploader)

What this tool does

Watches four input folders for product photos, uses AI to detect and crop individual items, transforms them with professional studio photography, then automatically creates draft WooCommerce products with names, descriptions, brand stories, pricing, SKUs, inventory controls, and dual images — finishing with an email notification.

Open source — MIT License — contributions welcome

What's New in Version 3

Version 3 adds Loot Bag (cross-body bag) support, per-product-type dimensions, inventory controls, and several bug fixes.

Change	Details
Loot Bag support (input/mobile/)	New product type for small cross-body bags. renderApron() now handles both aprons and loot bags with type-specific prompts. The loot bag prompt instructs gpt-image-2 to show the bag on a mannequin in cross-body style with randomized contents (Kindle, phone, water, Diet Coke) and sunglasses/cards in the front pocket.
Per-type dimensions in listing	woocommerce.js uses a switch on forcedType to output correct dimensions: bags (14"x15"x5"), aprons (bib height 30", waist width 31", strap length 31"), loot bags (5"x7.5" with front pocket description).
Inventory controls	All products created with manage_stock:true, stock_quantity:1, sold_individually:true. Ensures each unique item can only be sold once and limits cart quantity to 1.
Brand story bug fix	Fixed quality field logic: itemWord === "Shopping Bag" was always false since itemWord is "bag". Corrected to itemWord === "bag" so shopping bags correctly include reinforced handles; aprons and loot bags include front pocket.
Karis link guarantee	Purpose field post-processed via regex after AI response to ensure Karis Support Society is always a clickable hyperlink regardless of how the AI worded the sentence.
detectTag/replaceTag removed	gpt-image-2 reliably preserves tag text without special tag handling code. These functions were removed as no longer needed, simplifying the codebase.
PRICE_MOBILE added	New .env variable for loot bag pricing (default: \$15).

1. Project Overview

The ArtsyPhartsy Bag Uploader is a Node.js automation tool running on WSL. Drop a photo into an input folder and the tool handles AI detection, image transformation, and WooCommerce draft listing creation automatically.

1.1 Project Structure

File	Purpose
src/index.js	Folder watcher, queue manager, graceful shutdown on Ctrl+C
src/pipeline.js	Orchestrates the 4-step pipeline per photo
src/ai.js	All OpenAI API calls: detect, crop, puff, render, brand story, background replacement
src/woocommerce.js	Name uniqueness, image upload with retry, product creation with inventory controls and per-type dimensions
src/email.js	Notification email when drafts are ready
src/logger.js	Winston logging to console and file
.env	All credentials and configuration — never commit to git
logs/uploader.log	Full run history for debugging
README.md	Setup instructions and photography tips

1.2 Quick Start

```
cd artsy-phartsy-uploader
cp .env.example .env    # fill in your credentials
npm install
npm start
```

Drop a photo into one of the four input folders. Press Ctrl+C to stop gracefully.

1.3 Input Folders and Pricing

Folder	Item Type	Default Price
input/regular/	Light fabric tote bags	\$20.00
input/heavy/	Heavy fabric tote bags	\$25.00
input/aprons/	Aprons	\$20.00
input/mobile/	Loot bags (cross-body)	\$15.00

Configuring Prices
All prices are set in .env via PRICE_HEAVY, PRICE_LIGHT, PRICE_APRON, and PRICE_MOBILE. No code changes needed to adjust pricing.

2. The Four-Step Pipeline

Every photo goes through these four steps. Each step has fallback behaviour — failures are logged and processing continues rather than crashing.

Step 1 — AI Item Detection (`detectBags` in `ai.js`)

The photo is sent to GPT-4o (high detail) which returns for each item:

- A creative "Miss ____" product name reflecting the fabric pattern and vibe
- A 1-2 sentence product description highlighting the upcycled fabric
- Normalized bounding box coordinates (x, y, width, height as 0.0-1.0 fractions)

The itemWord passed to GPT-4o matches the product type: "bag", "apron", or "mobile". Each name is checked for uniqueness via a whimsical variant chain:

- "Miss X" > "Miss X Too" > "Miss X's Sister" > "Miss X's Twin" > "Miss X's Cousin" > "Miss X's Doppelganger" > "Miss X's Long Lost Cousin"
- If all variants are taken, GPT-4o generates a completely new name

Step 2 — Smart Cropping (`cropBags` in `ai.js`)

EXIF Auto-Rotation

`sharp().rotate()` applies phone EXIF orientation before cropping so items photographed in portrait mode always appear correctly oriented.

Smart Per-Side Padding

Each side's padding is constrained to 45% of the gap to the nearest neighbouring bounding box, preventing crops from bleeding into adjacent items in batch photos.

```
padding per side = min(10%, gapToNeighbour x 0.45)
```

Photography Tip

Leave 2-3 inches of backdrop between items in batch photos. When items touch or overlap, bounding boxes may themselves overlap and no padding adjustment can fix the resulting bleed-through.

Step 3a — Bag Puffing (`puffBag` in `ai.js`)

The cropped flat bag is sent to `gpt-image-2` to produce a filled 3D product shot. The prompt instructs the model to:

- Fill the bag with volume as if stuffed with a pillow
- Keep all fabric patterns, colors, and textures exactly as they appear
- Preserve the branding tag at the top center between handles
- Apply a randomly selected presentation style — 6 options
- Use a clean neutral light gray studio background

Presentation Variety — 6 Styles

- Upright with handles standing tall
- Upright with handles flopped forward over the front
- Slightly angled left with handles standing
- Slightly angled right with one handle flopped, one standing
- Upright with both handles drooping to each side
- Three-quarter angle with handles flopped forward

Each style also instructs the model to preserve the handle attachment position from the original photo.

Step 3b — Apron & Loot Bag Rendering (`renderApron` in `ai.js`)

Aprons

`gpt-image-2` places the apron on a professional faceless dress form mannequin. The bow is tied at the back (not visible), the bib sits flat against the chest, and the rendered length is 5 inches shorter than the flat photo.

Loot Bags (Cross-Body)

`gpt-image-2` shows the bag on a mannequin in cross-body style. The prompt specifies:

- Strap draped over one shoulder (randomized left/right), bag resting on opposite hip
- Only the front side of the strap visible — other side goes behind the mannequin
- Opening at top reveals a randomized stored item: Kindle, smartphone, water bottle, or Diet Coke
- Front pocket shows sunglasses or cards (empty if bag contains a beverage)
- Bag size and pocket position remain unchanged from original photo

NEW / CHANGED

NEW in v3: Loot bag cross-body mannequin rendering with detailed prompt and randomized contents for visual variety across listings.

Step 3c — Background Replacement (`prepareSecondaryImage` in `ai.js`)

The original flat crop is sent to `gpt-image-2` to replace the backdrop with clean neutral light gray (`#f0f0f0`), matching the hero image. Falls back to a color-boosted crop if the API call fails.

Step 4 — WooCommerce Upload (`woocommerce.js`)

- Primary image (puffed/rendered) uploaded to WordPress media library
- 1500ms delay after primary upload — WordPress needs time to process before image ID can be attached to a product
- Upload retries up to 3x on `ETIMEDOUT` or 5xx errors with 3s/6s/9s progressive delays
- If primary upload fails after all retries, secondary upload and product creation are both skipped
- Secondary image (background-replaced flat crop) uploaded
- Brand story generated by GPT-4o with slight variation per item
- Draft product created with name, description, brand story, price, SKU, category, images, and inventory controls

NEW / CHANGED

NEW in v3: All products created with `manage_stock:true`, `stock_quantity:1`, `sold_individually:true` — ensures each unique item can only be purchased once and limits cart quantity to 1.

3. Key Files in Detail

3.1 ai.js — All AI Operations

Function	What it does
detectBags()	GPT-4o high-detail: finds items, names them "Miss ____", returns bounding boxes. itemWord matches product type.
cropBags()	sharp: EXIF rotate, smart-pad crop each item. Padding constrained by neighbour proximity.
generateAlternativeName()	GPT-4o: generates fresh name when all 6 whimsical variants are taken
generateBrandStory()	GPT-4o: varied brand story per item type. Validates returned field names. Guarantees Karis hyperlink.
puffBag()	gpt-image-2: 3D puffed tote bag with random presentation style
renderApron()	
prepareSecondaryImage()	gpt-image-2: apron on dress form OR loot bag on mannequin cross-body — selected by forcedType
generateSKU()	gpt-image-2: background replacement to studio gray. Falls back to color-boost on failure.
	Unique SKU in format AP-XXXXXX-YYYYZZ

3.2 woocommerce.js — Store Integration

Function	What it does
checkNameExists()	Searches WooCommerce for exact name match across all product statuses
ensureUniqueName()	Runs whimsical variant chain until unique name found; returns null if all exhausted
uploadImage()	Uploads to WordPress media library with auto-compression and 3x retry logic
createDraftProduct()	Builds product data with type-specific dimensions and inventory controls; creates WooCommerce draft
uploadBag()	Orchestrates image uploads then product creation; aborts cleanly if primary image fails
getDraftsUrl()	Returns WP Admin drafts URL for use in notification email

3.3 Product Dimensions by Type

Product Type	Dimensions shown in listing
Bag (regular/heavy)	14 inches wide, 15 inches high and 5 inches deep
Apron	Height (top of bib to bottom): 30 inches, Width (between waist straps): 31 inches, Waist strap length: 31 inches each
Loot Bag (mobile)	5 inches wide, 7.5 inches high. Suitable for water, smartphone,

Kindle, or other loot. Front pocket for sunglasses, keys, cards, etc.

3.4 Brand Story HTML Structure

Each product's short_description (right panel above Add to Cart) is structured as:

```
<p>{AI-generated item description}</p>
<ul>
  <li><strong>Occupation:</strong> Shopping Bag | Apron | Loot Bag</li>
  <li><strong>Handmade by:</strong> {varied text}</li>
  <li><strong>Handmade from:</strong> {varied text}</li>
  <li><strong>Quality:</strong> {varied – reinforced handles for bags; front pocket
for aprons/loot bags}</li>
  <li><strong>Purpose:</strong> {varied} <a href="...">Karis Support Society</a></li>
  <li><strong>[Type] Dimensions:</strong> {type-specific dimensions}</li>
</ul>
```

Brand Story Validation

generateBrandStory() validates all 5 expected field names after parsing. If any are missing (AI occasionally renames fields), it falls back to hardcoded defaults. The Karis link is post-processed via regex to ensure it is always a clickable hyperlink.

4. Environment Variables (.env)

Copy .env.example to .env and fill in your values. Never commit this file — it is listed in .gitignore.

Variable	Purpose
OPENAI_API_KEY	OpenAI API key — needs access to GPT-4o and gpt-image-2
WOO_URL	WooCommerce store URL (e.g. https://yourstore.com)
WOO_CONSUMER_KEY	WooCommerce REST API consumer key (ck_...)
WOO_CONSUMER_SECRET	WooCommerce REST API consumer secret (cs_...)
WP_USERNAME	WordPress username for media uploads
WP_APP_PASSWORD	WordPress Application Password (Users > Profile > Application Passwords in WP Admin)
SMTP_HOST	Email server hostname
SMTP_PORT	Email port (587 for TLS)
SMTP_USER	Sending email address
SMTP_PASS	Email password or app password
NOTIFY_EMAILS	Comma-separated notification recipients
PRICE_HEAVY	Heavy bag price in dollars (default: 25)
PRICE_LIGHT	Light bag price in dollars (default: 20)
PRICE_APRON	Apron price in dollars (default: 20)
PRICE_MOBILE	Loot bag price in dollars (default: 15)
INPUT_FOLDER	Root input folder path (default: ./input)
OUTPUT_FOLDER	Processed output folder path (default: ./output)
FILE_SETTLE_MS	Wait time after file detected before processing (default: 2000)

Two Authentication Systems

WooCommerce REST API uses WOO_CONSUMER_KEY + WOO_CONSUMER_SECRET via Basic Auth. WordPress media upload uses a separate Application Password (WP_APP_PASSWORD) — these are different authentication systems on the same WordPress install.

5. Key Design Decisions

5.1 gpt-image-2 vs gpt-image-1

The project originally used gpt-image-1. Despite extensive prompt engineering, it consistently generated garbled tag text. Switching to gpt-image-2 (available April 2026) resolved the issue immediately — its reasoning step means it processes the tag preservation instruction contextually rather than generating plausible-looking but wrong text.

The reasoning step also eliminated the need for `detectTag()` and `replaceTag()` helper functions, simplifying the codebase significantly.

5.2 Draft Products, Not Published

All products are created as drafts. The store owner reviews each listing before publishing — useful since AI occasionally makes naming or description mistakes, and to verify batch photo separation worked correctly.

5.3 Inventory Controls

Each item is created with `manage_stock:true`, `stock_quantity:1`, `sold_individually:true`. This reflects that every item is unique and handmade, can only be sold once, and prevents customers from adding more than one to their cart.

5.4 The Image Upload Race Condition

WordPress returns a media ID immediately on upload but continues processing in the background. Creating a product too quickly causes WordPress to reject the ID as invalid. A 1500ms delay after primary upload prevents this reliably.

5.5 Smart Padding vs Fixed Padding

Fixed padding caused crops to bleed into neighbours in batch photos. The smart algorithm constrains each side to 45% of the gap to the nearest neighbour — single items get generous padding, tightly-packed items get proportionally less, automatically.

5.6 Brand Story Field Validation

GPT-4o occasionally returns JSON with unexpected field names. Validation catches this and falls back to hardcoded defaults rather than allowing "undefined" to appear on product listings.

6. API Costs

Operation	Model	Approx. Cost
Item detection + naming	GPT-4o (high detail)	~\$0.01
Brand story generation	GPT-4o	~\$0.005
Puffing OR mannequin rendering	gpt-image-2	~\$0.03-0.04
Background replacement	gpt-image-2	~\$0.03-0.04
Total per item	—	~\$0.07-0.10

For a backlog of 100 items, expect approximately \$7-10 in OpenAI API costs.

7. Known Limitations

7.1 Tightly-Packed Batch Photos

When items are photographed very close together, GPT-4o bounding boxes may overlap, causing crops to bleed. Recommendation: leave 2-3 inches of backdrop between items.

7.2 Background Replacement Accuracy

Works best when the item has good contrast against the original backdrop. If item fabric is the same color as the background, edge detection may be inaccurate.

7.3 Loot Bag Contents Randomization

The AI is instructed to randomize contents and shoulder side. It follows this reasonably well but may occasionally repeat the same choice across consecutive items.

7.4 Network Errors

Image uploads retry up to 3 times on timeout or server errors. Database connection errors from the hosting platform are not retryable — photos go to processed/failed/ for manual reprocessing.

7.5 AI Reliability

AI models are probabilistic. Occasional unexpected behaviours: brand story returning different field names (handled by validation), puffing failing on connection error (falls back to flat crop), bounding boxes slightly inaccurate on complex batch photos. All known failure modes have fallback behaviour.

8. Troubleshooting

8.1 Checking the Log

```
tail -50 ~/artsy-phartsy/logs/uploader.log
grep "Miss Bag Name" ~/artsy-phartsy/logs/uploader.log
```

8.2 Reprocessing Failed Photos

Photos that failed are moved to input/[folder]/processed/failed/. Copy back to retry:

```
cp ~/artsy-phartsy/input/heavy/processed/failed/photo.jpg ~/artsy-phartsy/input/heavy/
```

8.3 Common Errors

Error	Fix
unsupported mimetype (application/octet-stream)	Tmp PNG path regex failed. Verify croppedPath has a standard file extension.
ETIMEDOUT on image upload	Auto-retries 3x with progressive delays. If all fail, photo goes to processed/failed/.
Error establishing a database connection	WordPress hosting issue. Wait a few minutes and reprocess.
#xxx is an invalid image ID	Race condition — increase the 1500ms delay in uploadBag() if this recurs.
Brand story fields showing "undefined"	AI returned unexpected field names. Check logs for "Brand story missing fields" warning.
Puffing failed — using cropped original	OpenAI connection error. Reprocess if puffed image is needed.

8.4 Verify Clean Deployment

```
grep "gpt-image" ~/artsy-phartsy/src/ai.js
```

Should only show gpt-image-2.

```
grep -n "blur" ~/artsy-phartsy/src/ai.js
```

Should return nothing.

9. Dependencies

Package	Version	Purpose
openai	latest	GPT-4o + gpt-image-2 API client
sharp	latest	Image processing: rotate, crop, resize, compress, composite
chokidar	latest	Cross-platform file system watcher
axios	latest	HTTP client for WooCommerce and WordPress REST APIs
form-data	latest	Multipart form data for WordPress media upload
nodemailer	^8.0.8	Email notifications
winston	latest	Structured logging to console and file
dotenv	latest	Loads .env file into process.env

Node.js Requirement

Requires Node.js 20+ installed via nvm on WSL. Must use the WSL Linux node binary — not Windows node.exe on the PATH. Verify with: `which node`. If sharp fails to install, this is usually the cause.

10. Photography Guide

With AI background replacement, the focus shifts to lighting and framing rather than backdrop choice.

10.1 Lighting

- Bright even lighting is the most important factor
- Natural daylight near a window on an overcast day is ideal — clouds act as a giant softbox
- Avoid direct harsh sunlight — creates shadows that confuse edge detection
- Multiple soft light sources from different angles work well indoors

10.2 Framing — All Products

- Leave space above and below the item in every shot
- For batch photos: leave 2-3 inches of backdrop between each item
- Maximum 3-4 items per batch photo for accurate AI separation
- The full item must be visible in frame

10.3 Loot Bag Photography

- Photograph flat, showing the front face clearly
- Ensure the front pocket is visible and its position is clear
- The shoulder strap should be visible in the photo
- The opening at the top should be clearly visible

10.4 Background

- Background choice matters much less now — AI replaces it automatically
- Good contrast between item and backdrop still helps edge detection
- Avoid photographing a dark item against a dark background

11. Quick Reference

Daily Operations

- Start: `cd ~/artsy-phartsy && npm start`
- Drop photos into `input/regular/`, `input/heavy/`, `input/aprons/`, or `input/mobile/`
- Watch the log: `tail -f logs/uploader.log`
- Email arrives when drafts are ready — review and publish in WP Admin
- Stop: `Ctrl+C` (waits for current job to finish)

AI Models Used

- GPT-4o — detection, naming, brand story generation
- gpt-image-2 — puffing, mannequin rendering, background replacement
- gpt-image-2 reasoning capability reliably preserves branding tag text — no special tag handling code needed

Verify Clean Deployment

- `grep "gpt-image" src/ai.js` — should only show `gpt-image-2`
- `grep -n "blur" src/ai.js` — should return nothing